

IA en el Desarrollo de Software

- Las herramientas de **inteligencia artificial (IA)** desempeñan un papel cada vez más importante en el desarrollo de software.
- La IA se utiliza para generar código nuevo, revisar y probar el código y las aplicaciones existentes, ayudar a los equipos a implementar continuamente nuevas características y mucho más.
- Las soluciones de IA no sustituyen a los equipos de desarrollo humano. Más bien, estas herramientas se utilizan para mejorar el proceso de desarrollo, creando equipos más productivos y software más sólido.

IA en el Desarrollo de Software

Generación de código

La IA generativa puede crear fragmentos de código y funciones completas basándose en instrucciones del lenguaje natural o en el contexto del código.

Autocompletado

Cuando un desarrollador está escribiendo código, las herramientas de IA generativa pueden analizar el código escrito y su contexto y sugerir la siguiente línea de código.

Si procede, el desarrollador puede aceptar esta sugerencia. Permite ahorrar tiempo al desarrollador y también puede ser una herramienta útil para los desarrolladores que trabajan en lenguajes de codificación en los que no tienen mucha experiencia o con los que hace tiempo que no trabajan.

IA en el Desarrollo de Software

Escritura de código original

Los desarrolladores pueden dar instrucciones directamente a las herramientas de IA con instrucciones específicas en lenguaje sencillo. Estas instrucciones incluyen especificaciones como el lenguaje de programación, la sintaxis y lo que el desarrollador quiere que haga el código. Las herramientas de IA generativa pueden producir entonces un fragmento de código o una función completa; los desarrolladores revisan entonces el código haciendo ediciones cuando es necesario. Estas correcciones ayudan a entrenar aún más el modelo.

Traducción de código y modernización de aplicaciones

Las herramientas de IA generativa pueden traducir el código de un lenguaje de programación a otro, ahorrando tiempo a los desarrolladores y reduciendo el riesgo de errores manuales. Esto es útil a la hora de modernizar aplicaciones, por ejemplo, al traducir COBOL a Java.

La generación de código con IA también puede ayudar a automatizar la codificación repetitiva involucrada en la migración de infraestructuras o software tradicionales a la nube.

IA en el Desarrollo de Software

Pruebas

Los desarrolladores pueden dar instrucciones a las herramientas de IA generativa para que construyan y realicen pruebas en fragmentos de código existentes.

Además, gracias a su capacidad para analizar grandes conjuntos de datos, las herramientas de IA pueden descubrir patrones y anomalías en los datos que pueden utilizarse para encontrar posibles problemas. Cuando las herramientas de IA descubren problemas, ya sea mediante pruebas o monitorización, pueden automatizar la corrección de errores y fallos.

Implementación

Los desarrolladores pueden utilizar herramientas de IA para ayudar a gestionar los cambios en el código realizados a lo largo del ciclo de vida de desarrollo del software y asegurarse de que dichos cambios se implementan correctamente.

Además, las herramientas de IA ayudan a los desarrolladores a implementar nuevas características de manera fluida integrando el nuevo código en los entornos de producción sin interrumpir el servicio. También pueden actualizar automáticamente la documentación después de que se hayan realizado cambios en el software.

Arquitectura de Software

La arquitectura de software se refiere a la estructura organizativa y conceptual que subyace en el diseño y desarrollo de sistemas de software.

Es el proceso de definir una solución estructural que cumple con todos los requisitos funcionales y no funcionales, como la escalabilidad, el rendimiento y la seguridad.

Su importancia radica en su capacidad para proporcionar un marco de trabajo que guía las decisiones de diseño y desarrollo. Aquí, se establece el enfoque, la estructura, y las tecnologías necesarias para construir una aplicación.

Arquitectura de Software

“La arquitectura es un nivel de diseño que hace foco en aspectos "más allá de los algoritmos y estructuras de datos de la computación; el diseño y especificación de la estructura global del sistema es un nuevo tipo de problema “ — “

An introduction to Software Architecture" de David Garlan y Mary Shaw

“La Arquitectura de Software se refiere a las estructuras de un sistema, compuestas de elementos con propiedades visibles de forma externa y las relaciones que existen entre ellos. “

Software Engineering Institute (SEI)

Si bien todas las definiciones son diferentes, todas coinciden en que la arquitectura se centra en la estructura del sistema, los componentes que lo conforman y la relación que existe entre ellos.

Arquitectura de Software

La Arquitectura de Software es la estructura fundamental de un sistema de software.

Define:

- componentes,
- módulos,
- relaciones,
- interacción entre partes del sistema,
- tecnologías utilizadas.

La arquitectura actúa como el “plano” del software.

Tipos de Arquitectura de Software

Monolítica

Arq. Orientada aServicios -SOA

Microservicios

Cliente-Servidor

Arq. en Capas

DevOps

Tipos de Arquitectura de Software

Arquitectura Monolítica

En una aplicación monolítica, todo se construye como una unidad con una única base de código.

Por ejemplo, una aplicación web empresarial que gestione los pedidos de los clientes, la gestión del inventario y la facturación tendría todas estas funciones estrechamente integradas. Cambiar una parte requiere entender cómo afecta a todas las demás.

Aunque esta simplicidad facilita el desarrollo inicial del software, genera problemas a medida que las aplicaciones crecen. Cualquier cambio requiere reconstruir y volver a implementar toda la aplicación. Si se necesita más capacidad en una arquitectura monolítica, hay que escalar toda la aplicación, incluso las partes que no requieren una mayor capacidad de rendimiento.

Tipos de Arquitectura de Software

Arquitectura Monolítica

Todo el sistema funciona como una única aplicación. Todos los componentes están interconectados y se despliegan como una única unidad.

Ventajas

- Simple de desarrollar,
- Menor complejidad inicial.

Desventajas

- Difícil mantenimiento,
- Poca escalabilidad.

Tipos de Arquitectura de Software

Arquitectura Orientada a Servicios-SOP

Las Aplicaciones Orientadas a Servicios (SOA - Service Oriented Architecture) organizan funcionalidades en servicios independientes que pueden comunicarse entre sí. Cada servicio cumple una función específica.

Los servicios utilizan estándares de interfaz comunes y un patrón arquitectónico para que puedan incorporarse rápidamente a nuevas aplicaciones.

Ejemplo

En una tienda online:

- servicio de pagos,
- servicio de usuarios,
- servicio de productos,
- servicio de envíos.

Cada uno funciona de forma independiente.

Tipos de Arquitectura de Software

Arquitectura Orientada a Servicios-SOP

Características

- reutilización,
- interoperabilidad,
- independencia,
- integración entre sistemas.

Ventajas

- Mayor agilidad empresarial, tiempo de comercialización más rápido:
- Capacidad para utilizar funciones heredadas en nuevos mercados:
- Mejora de la colaboración entre la empresa y la TI
- Mejora de la colaboración entre la empresa y la TI

Tipos de Arquitectura de Software

Arquitectura Orientada a Servicios-SOP

Aplicaciones Orientadas a Servicios (SOA y APIs)

Es un estilo de diseño de software donde los componentes de la aplicación entregan servicios a otros componentes a través de un protocolo de comunicación en una red.

Conceptos Clave

- **SOA (Service-Oriented Architecture):** Un enfoque de diseño que permite que diferentes aplicaciones se comuniquen entre sí mediante servicios interconectados (históricamente muy asociado a tecnologías como SOAP y XML).
- **APIs (Application Programming Interfaces):** El puente moderno de comunicación. Hoy en día, el estándar de la industria es **REST** (que utiliza JSON para el intercambio de datos debido a su ligereza) y, para necesidades de tiempo real o alta eficiencia, **gRPC** o **GraphQL**.

Tipos de Arquitectura de Software

Arquitectura Cliente/Servidor

La arquitectura cliente-servidor es un modelo de diseño de software en red donde las tareas se dividen entre el **cliente** (que solicita recursos o servicios) y el **servidor** (que los procesa y proporciona). Entre ambos se tiene que efectuar una comunicación de red, habitualmente mediante Internet.

Ejemplos:

- El cliente sería el navegador, y el servidor sería la máquina donde están instaladas las aplicaciones, las bases de datos y otros recursos remotos.
- Servicio de email.
- Servicio bancario.
- Aplicaciones móviles conectados a servicios en la nube.

Tipos de Arquitectura de Software

Arquitectura Cliente/Servidor

Ventajas

- Centralización de recursos y seguridad.
- Mantenimiento y actualizaciones más fáciles.
- Escalabilidad para adaptarse al crecimiento del sistema
- Mayor control sobre el flujo de datos
- Reducción de redundancias en la gestión de datos

Desventajas

- Dependencia en el servidor
- Costes de mantenimiento
- Congestión en la red

Tipos de Arquitectura de Software

Arquitectura en Capas

La arquitectura de tres capas es una arquitectura de aplicaciones de software bien establecida que organiza las aplicaciones en tres capas lógicas y físicas: la capa de presentación, o interfaz de usuario; la capa de aplicación, donde se procesan los datos; y la capa de datos, donde se almacenan y gestionan los datos de la aplicación.

El principal beneficio de la arquitectura de tres niveles es que, dado que cada nivel se ejecuta en su propia infraestructura, cada nivel puede ser desarrollado simultáneamente por un equipo de desarrollo independiente. Y puede actualizarse o escalarse según sea necesario sin afectar a los otros niveles.

Tipos de Arquitectura de Software

Arquitectura en Capas

Ventajas

- Separación lógica y física de la funcionalidad.
- Desarrollo más rápido
- Escalabilidad mejorada
- Mayor fiabilidad
- Seguridad mejorada

Tipos de Arquitectura de Software

Arquitectura en Capas

Desventajas

- Complejidad
- Tiempo y Esfuerzo de desarrollo
- Consumo de recursos
- Costos
- Rendimiento

Tipos de Arquitectura de Software

Microservicios

El sistema se divide en pequeños servicios independientes que se comunican entre si. Cada servicio gestiona una función empresarial específica.

Ejemplo

•Empresas como Netflix, Uber, Amazon, Spotify y Airbnb utilizan microservicios para gestionar millones de usuarios y transacciones cada día.

Ventajas

- escalabilidad,
- mantenimiento independiente, implementar actualizaciones de forma aislada en cada servicio.
- flexibilidad tecnológica.
- recuperarse fácilmente de los fallos,resilencia.

Tipos de Arquitectura de Software

Microservicios

Desventajas

- Complejidad de pruebas
- Consideraciones sobre seguridad
- Coordinación de la comunicación
- Gestión de datos

Tipos de Arquitectura de Software

Microservicios

Los microservicios resuelven los problemas planteados en la Arquitectura Monolítica dividiendo la aplicación en servicios separados que se comunican entre sí a través de interfaces de programación de aplicaciones (API). Cada servicio tiene su propio código base y base de datos y puede implementarse de forma independiente.

Por ejemplo la plataforma musical Spotify, que cuenta con cientos de millones de usuarios en todo el mundo. Cuando la demanda de música en streaming aumenta durante el lanzamiento de un álbum importante, Spotify amplía sus servicios de entrega de audio y listas de reproducción sin afectar a la autenticación de los usuarios ni al procesamiento de pagos. Este enfoque permite que diferentes equipos trabajen en servicios individuales al mismo tiempo sin conflictos.

Tipos de Arquitectura de Software

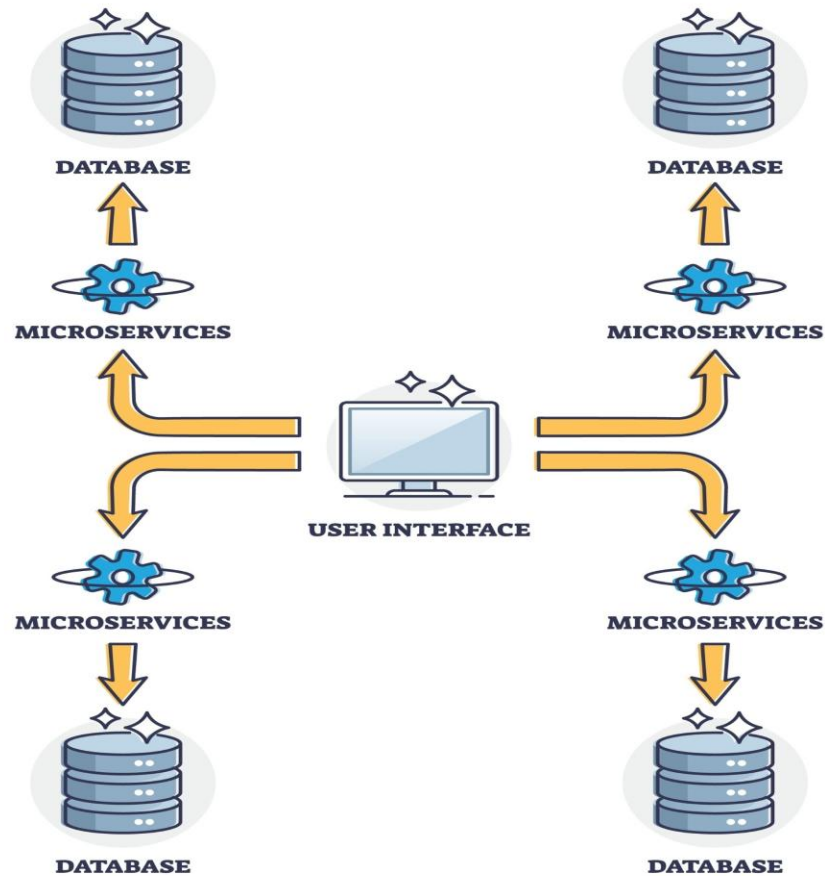
Microservicios - Funcionamiento

Los microservicios funcionan mediante una combinación de patrones arquitectónicos, métodos de comunicación y tecnologías de apoyo. Cada microservicio se ejecuta como un proceso propio y presenta sus características a través REST API, colas de mensajes o flujos de eventos. Los servicios se comunican a través de la red para compartir datos y desencadenar acciones.

Por ejemplo, cuando un cliente pide comida a través de Uber Eats, varios servicios trabajan juntos en secuencia. El sistema verifica la disponibilidad del restaurante, procesa el pago, asigna un controlador de entrega y envía actualizaciones al cliente. Una pasarela API suele gestionar esta comunicación, actuando como un único punto de entrada que enruta las solicitudes a los microservicios adecuados.

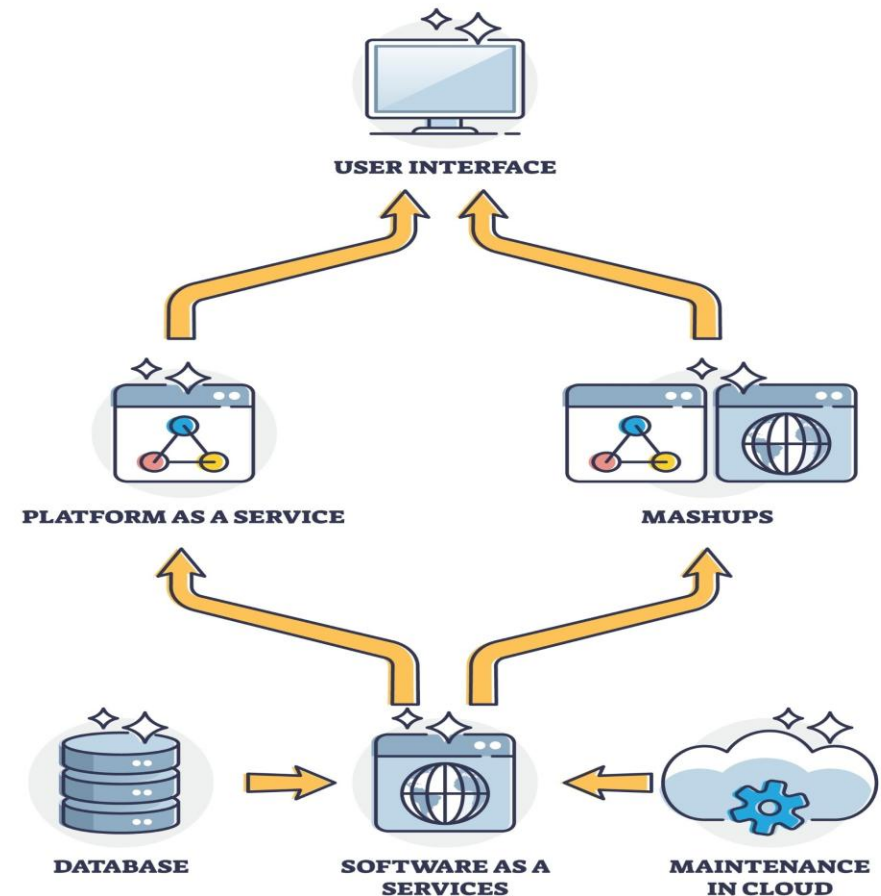
Tipos de Arquitectura de Software

MICROSERVICES



VS

SOA



Introducción a DevOps

DevOps es una metodología de desarrollo de software que acelera la entrega de aplicaciones y servicios de alto rendimiento al combinar y automatizar el trabajo de los equipos de desarrollo (Dev) y operaciones de TI (Ops).

DevOps esboza tanto un proceso de desarrollo de software como un cambio de cultura organizativa que fomenta la coordinación y la colaboración entre los equipos de desarrollo y operaciones de TI. Tradicionalmente, estos dos grupos trabajaban por separado en silos distintos.

En la metodología DevOps, trabajan como un solo equipo con un conjunto de herramientas y prácticas comunes.

Introducción a DevOps

Tradicionalmente, los desarrolladores (Dev) escribían el código y el equipo de operaciones (Ops) se encargaba de subirlo al servidor y mantenerlo vivo. Esto generaba fricción (*"En mi máquina funciona, es problema de operaciones"*).

DevOps es una cultura, movimiento y conjunto de prácticas que unifica el desarrollo de software y la operación del software.

DevOps es una metodología que integra:

- desarrollo de software (Development),
- operaciones de sistemas (Operations).

Busca mejorar:

- colaboración,
- automatización,
- velocidad de entrega,
- calidad del software.

Introducción a DevOps

Ciclo DevOps

1. Planificación
2. Desarrollo
3. Integración
4. Pruebas
5. Despliegue
6. Monitoreo

Herramientas DevOps

Integración Continua

- Jenkins
- GitHub Actions

Contenedores

- Docker

Orquestación

- Kubernetes

Monitoreo

- Prometheus
- Grafana

Objetivos de DevOps

- integración continua,
- despliegue continuo,
- automatización,
- reducción de errores,
- entrega rápida.

Introducción a DevOps

Prácticas Esenciales de DevOps

- **CI/CD (Integración Continua / Despliegue Continuo):** Automatizar el proceso. Cada vez que un programador sube código, un servidor automático lo prueba (CI) y, si todo está bien, lo sube a producción (CD) sin intervención manual.
- **Infraestructura como Código (IaC):** Configurar servidores, bases de datos y redes mediante archivos de texto (código) en lugar de hacer clics manuales en paneles de control (herramientas como Terraform).
- **Contenedores (Docker):** Permiten empaquetar la aplicación junto con todas sus dependencias (librerías, sistema operativo específico) en un "contenedor". Esto garantiza que el software corra **exactamente igual** en la computadora del desarrollador y en el servidor de producción, eliminando el clásico problema de compatibilidad.

Introducción a DevOps

Beneficios de DevOps

Beneficio	Descripción
Entregas rápidas	Software más frecuente
Automatización	Menos tareas manuales
Calidad	Reducción de errores
Colaboración	Mejor trabajo en equipo
Escalabilidad	Sistemas más flexibles